Informe de Avance: Intérprete de Lenguaje Personalizado

1. Introducción y Objetivos

El objetivo de este proyecto es construir un intérprete para un lenguaje de programación imperativo propio, con soporte nativo para operaciones matriciales y números reales.

El lenguaje permite la declaración explícita de variables, manipulación de matrices de hasta 300x300 elementos , y estructuras de control estándar como selecciones (`IF-THEN-ELSE`) y repeticiones (`WHILE`).

2. Definición de Componentes Léxicos (Tokens)

Se han identificado y definido los siguientes componentes léxicos necesarios para cumplir con los requisitos del lenguaje.

2.1. Palabras Reservadas

Token	Lexema	Descripción	Origen del Requisito
TOKEN_PROGRAM	program	Inicio del programa	Estándar
TOKEN_VAR	var	Sección de declaraciones	
TOKEN_BEGIN	begin	Inicio de bloque/cuerpo	
TOKEN_END	end	Fin de bloque/cuerpo	
TOKEN_IF	if	Condicional Si	
TOKEN_THEN	then	Entonces	
TOKEN_ELSE	else	Sino	
TOKEN_WHILE	while	Ciclo Mientras	
TOKEN_DO	do	Hacer (para el ciclo)	Estándar
TOKEN_READ	read	Lectura por teclado	
TOKEN_WRITE	write	Escritura en pantalla	
TOKEN_TYPE_REAL	real	Tipo de dato real	
TOKEN_TYPE_MATRIX	matrix	Tipo de dato matriz	

2.2. Operadores y Símbolos

Token	Lexema	Descripción	Origen del Requisito
TOKEN_ASIG	:=	Asignación	
TOKEN_MAS	+	Suma (aritmética/matricial)	
TOKEN_MENOS	-	Resta	
TOKEN_POR	*	Producto	
TOKEN_DIV	/	División	
TOKEN_MENOR	<	Operador relacional	
TOKEN_MAYOR	>	Operador relacional	
TOKEN_MENOR_IGUAL	<=	Operador relacional	
TOKEN_MAYOR_IGUAL	>=	Operador relacional	
TOKEN_IGUAL	“=”	Operador relacional	
TOKEN_DISTINTO	<>	Operador relacional	
TOKEN_PAREN_IZQ	(	Agrupación expresiones	Estándar
TOKEN_PAREN_DER	)	Agrupación expresiones	Estándar
TOKEN_COR_IZQ	[	Índices de matriz	
TOKEN_COR_DER	]	Índices de matriz	
TOKEN_COMA	,	Separador de listas	
TOKEN_PUNTO_COMA	;	Fin de sentencia	Estándar
TOKEN_DOS_PUNTOS	:	Definición de tipo	Estándar
TOKEN_PUNTO	.	Fin de programa	Estándar

2.3. Componentes Léxicos Complejos

Se definieron mediante Expresiones Regulares (ER) y se diseñaron sus Autómatas Finitos Determinísticos (AFD) correspondientes.

2.4. Especificación Formal de Expresiones Regulares

Para evitar ambigüedades en la notación, se presentan las Expresiones Regulares (ER) de los componentes léxicos complejos en dos formatos:

1. **Notación Extendida (Moderna):** Utiliza atajos comunes en herramientas de programación (`+` para "uno o más", `?` para "opcional", `[]` para rangos).

2. **Notación Clásica (Pura):** Utiliza estrictamente las tres operaciones fundamentales: Unión (`|`), Concatenación (implícita o `.`) y Clausura de Kleene (`*`).

Definiciones Auxiliares

Para facilitar la lectura de la notación clásica:

Letra (**L**) = `a|b|...|z|A|B|...|Z`

`Dígito (D) = 0|1|2|3|4|5|6|7|8|9`

- **IDENTIFICADOR (`TOKEN_ID`):**
  - **ER:** `[a-zA-Z_]([a-zA-Z0-9_])* o (L | _) (L | D | _)*`
  - **Descripción:** Comienza con letra o guion bajo, seguido de cero o más letras, dígitos o guiones bajos.
- **CONSTANTE REAL (`TOKEN_REAL`)**:
  - **ER:** `[0-9]+ \. [0-9]+ o D D* . D D*`
  - **Descripción:** Parte entera (al menos un dígito), seguida de un punto literal, seguida de la parte fraccionaria (al menos un dígito).
- **CONSTANTE ENTERA (`TOKEN_ENTERO`)**:
  - **ER:** `[0-9]+ o DD*`
  - **Descripción:** Secuencia de uno o más dígitos. Necesaria para definir dimensiones de matrices y accesos a índices.

3. Implementación del Analizador Léxico

El analizador léxico se implementó en **Free Pascal** utilizando una arquitectura de **Unit independiente** (**`AnalizadorLexico.pas`**) para modularidad.

Características Técnicas:

- **Manejo de Buffer:** Se implementó un sistema de lectura línea por línea (**`LineaActualStr`**) para facilitar el manejo de punteros y evitar problemas de "consumo excesivo" de caracteres al analizar tokens como números.
- **Despachador Central:** La función principal **`ObtenerSiguienteToken`** utiliza un **`case`** basado en el primer carácter leído para decidir rápidamente qué autómata específico ejecutar (**`EscanearIdentificador`**, **`EscanearNumero`** o procesamiento directo de símbolos), optimizando el rendimiento.

3.1. Código Fuente Actual (AnalizadorLexico.pas)

Parte 1: Interfaz y Definición de Tokens

```
unit AnalizadorLexico;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils;

type
  TTipoToken = (
    TOKEN_ERROR,
    TOKEN_EOF,
    TOKEN_ID,
    TOKEN_ENTERO,
    TOKEN_REAL,
    TOKEN_STRING,
    TOKEN_PROGRAM, TOKEN_VAR, TOKEN_BEGIN, TOKEN_END,
    TOKEN_IF, TOKEN_THEN, TOKEN_ELSE, TOKEN_WHILE, TOKEN_DO,
    TOKEN_READ, TOKEN_WRITE,
    TOKEN_TYPE_REAL, TOKEN_TYPE_MATRIX,
    TOKEN_MAS, TOKEN_MENOS, TOKEN_POR, TOKEN_DIV,
    TOKEN_ASIG, TOKEN_IGUAL, TOKEN_DISTINTO,
    TOKEN_MENOR, TOKEN_MAYOR, TOKEN_MENOR_IGUAL, TOKEN_MAYOR_IGUAL,
    TOKEN_PAREN_IZQ, TOKEN_PAREN_DER,
    TOKEN_COR_IZQ, TOKEN_COR_DER, TOKEN_LLAVE_IZQ, TOKEN_LLAVE_DER,
    TOKEN_COMA, TOKEN_PUNTO_COMA, TOKEN_DOS_PUNTOS, TOKEN_PUNTO
  );

var
  ArchivoFuente: TextFile;
  LineaActual: Integer = 0;

procedure InicializarLexico(RutaArchivo: String);
procedure FinalizarLexico;
function ObtenerSiguienteToken(var Lexema: String): TTipoToken;
```

Parte 2: Infraestructura de Lectura (Buffer)

```
implementation

var
  CaracterActual: Char = ' ';
  LineaActualStr: String = '';
  PosicionActual: Integer = 1;
  FinDeArchivo: Boolean = False;

procedure LeerSiguienteCaracter;
begin
  if FinDeArchivo then
  begin
    CaracterActual := #0;
    Exit;
  end;
  if PosicionActual > Length(LineaActualStr) then
  begin
    if EOF(ArchivoFuente) then
    begin
      FinDeArchivo := True;
      CaracterActual := #0;
      { Truco para asegurar que la última línea se procese si no tiene \n }
      if (Length(LineaActualStr) > 0) and (LineaActualStr[Length(LineaActualStr)] <> #10) then
         CaracterActual := #10
      else
         Exit;
    end
    else
    begin
      ReadLn(ArchivoFuente, LineaActualStr);
      LineaActualStr := LineaActualStr + #10;
      Inc(LineaActual);
      PosicionActual := 1;
    end;
  end;
  if FinDeArchivo and (CaracterActual = #0) then Exit;
  CaracterActual := LineaActualStr[PosicionActual];
  Inc(PosicionActual);
end;
```

Parte 3: Autómatas Específicos (Helpers)

```
procedure SaltarBlancos;
begin
  while (NOT FinDeArchivo) and (CaracterActual in [#9, #10, #13, ' ']) do
    LeerSiguienteCaracter;
end;

function EscanearIdentificador(var Lexema: String): TTipoToken;
var
  LexemaLower: String;
begin
  Lexema := '';
  while CaracterActual in ['a'..'z', 'A'..'Z', '0'..'9', '_'] do
  begin
    Lexema := Lexema + CaracterActual;
    LeerSiguienteCaracter;
  end;
  LexemaLower := LowerCase(Lexema);
  if LexemaLower = 'program' then Exit(TOKEN_PROGRAM)
  else if LexemaLower = 'var' then Exit(TOKEN_VAR)
  else if LexemaLower = 'begin' then Exit(TOKEN_BEGIN)
  else if LexemaLower = 'end' then Exit(TOKEN_END)
  else if LexemaLower = 'if' then Exit(TOKEN_IF)
  else if LexemaLower = 'then' then Exit(TOKEN_THEN)
  else if LexemaLower = 'else' then Exit(TOKEN_ELSE)
  else if LexemaLower = 'while' then Exit(TOKEN_WHILE)
  else if LexemaLower = 'do' then Exit(TOKEN_DO)
  else if LexemaLower = 'read' then Exit(TOKEN_READ)
  else if LexemaLower = 'write' then Exit(TOKEN_WRITE)
  else if LexemaLower = 'real' then Exit(TOKEN_TYPE_REAL)
  else if LexemaLower = 'matrix' then Exit(TOKEN_TYPE_MATRIX)
  else Result := TOKEN_ID;
end;

function EscanearNumero(var Lexema: String): TTipoToken;
var
  EsReal: Boolean;
begin
  Lexema := '';
  EsReal := False;
  while CaracterActual in ['0'..'9'] do
  begin
    Lexema := Lexema + CaracterActual;
    LeerSiguienteCaracter;
  end;
  if CaracterActual = '.' then
  begin
    Lexema := Lexema + CaracterActual;
    LeerSiguienteCaracter;
    if CaracterActual in ['0'..'9'] then
    begin
      EsReal := True;
      while CaracterActual in ['0'..'9'] do
      begin
        Lexema := Lexema + CaracterActual;
        LeerSiguienteCaracter;
      end;
    end
    else
    begin
      Result := TOKEN_ERROR;
      Exit;
    end;
  end;
  if EsReal then Result := TOKEN_REAL else Result := TOKEN_ENTERO;
end;
```

Parte 4: Inicialización y Despachador Principal (Inicio)

```
procedure InicializarLexico(RutaArchivo: String);
begin
  AssignFile(ArchivoFuente, RutaArchivo);
  Reset(ArchivoFuente);
  LineaActual := 0;
  PosicionActual := 1;
  LineaActualStr := '';
  FinDeArchivo := False;
  LeerSiguienteCaracter;
end;

procedure FinalizarLexico;
begin
  CloseFile(ArchivoFuente);
end;

function ObtenerSiguienteToken(var Lexema: String): TTipoToken;
begin
  SaltarBlancos;
  Lexema := '';
  if FinDeArchivo and (CaracterActual = #0) then Exit(TOKEN_EOF);

  case CaracterActual of
    'A'..'Z', 'a'..'z', '_': Result := EscanearIdentificador(Lexema);
    '0'..'9': Result := EscanearNumero(Lexema);
    '+': begin Lexema := '+'; Result := TOKEN_MAS; LeerSiguienteCaracter; end;
    '-': begin Lexema := '-'; Result := TOKEN_MENOS; LeerSiguienteCaracter; end;
    '*': begin Lexema := '*'; Result := TOKEN_POR; LeerSiguienteCaracter; end;
    '/': begin Lexema := '/'; Result := TOKEN_DIV; LeerSiguienteCaracter; end;
    '(': begin Lexema := '('; Result := TOKEN_PAREN_IZQ; LeerSiguienteCaracter; end;
    ')': begin Lexema := ')'; Result := TOKEN_PAREN_DER; LeerSiguienteCaracter; end;
    '[': begin Lexema := '['; Result := TOKEN_COR_IZQ; LeerSiguienteCaracter; end;
    ']': begin Lexema := ']'; Result := TOKEN_COR_DER; LeerSiguienteCaracter; end;
    '{': begin Lexema := '{'; Result := TOKEN_LLAVE_IZQ; LeerSiguienteCaracter; end;
    '}': begin Lexema := '}'; Result := TOKEN_LLAVE_DER; LeerSiguienteCaracter; end;
    ',': begin Lexema := ','; Result := TOKEN_COMA; LeerSiguienteCaracter; end;
    ';': begin Lexema := ';'; Result := TOKEN_PUNTO_COMA; LeerSiguienteCaracter; end;
    '.': begin Lexema := '.'; Result := TOKEN_PUNTO; LeerSiguienteCaracter; end;
```

Parte 5: Despachador Principal (Fin)

```
':': begin
           LeerSiguienteCaracter;
           if CaracterActual = '=' then begin Lexema := ':='; Result := TOKEN_ASIG;
LeerSiguienteCaracter; end
           else begin Lexema := ':'; Result := TOKEN_DOS_PUNTOS; end;
         end;
    '<': begin
           LeerSiguienteCaracter;
           if CaracterActual = '=' then begin Lexema := '<='; Result := TOKEN_MENOR_IGUAL;
LeerSiguienteCaracter; end
           else if CaracterActual = '>' then begin Lexema := '<>'; Result := TOKEN_DISTINTO;
LeerSiguienteCaracter; end
           else begin Lexema := '<'; Result := TOKEN_MENOR; end;
         end;
    '>': begin
           LeerSiguienteCaracter;
           if CaracterActual = '=' then begin Lexema := '>='; Result := TOKEN_MAYOR_IGUAL;
LeerSiguienteCaracter; end
           else begin Lexema := '>'; Result := TOKEN_MAYOR; end;
         end;
    '=': begin Lexema := '='; Result := TOKEN_IGUAL; LeerSiguienteCaracter; end;
    else begin Lexema := CaracterActual; Result := TOKEN_ERROR; LeerSiguienteCaracter; end;
  end; { fin del case }
end;

end.
```